

Análisis de complejidad computacional: ejercicios

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Analizar el costo de un algoritmo con la rutina completa: traza, patrón, sumatoria y fórmula cerrada.
- Contar ciclos con pasos distintos de uno, descendentes y con dependencias entre índices.
- Nombrar el ritmo de crecimiento de un costo con la notación O .

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA3: describir ideas con argumentos precisos y basados en evidencia.

Objetivos de aprendizaje II

Referencia bibliográfica

T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.^a ed., secciones 2.2 y 3.1, apéndice A (sección A.1); R. Thareja, *Data Structures Using C*, capítulo 2.

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento
- 7 Todo junto
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento
- 7 Todo junto
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Lo que dejó la semana I

- Cada instrucción simple cuenta 1; la condición de un `while` se evalúa una vez más que su cuerpo.
- Ciclos en secuencia suman; anidados independientes multiplican.
- Si el interno depende del externo: congelar i , contar el interno como función de i y sumar sobre i .
- Cuando el conteo depende de los datos, se reporta el peor caso.

El formulario I

Fórmulas cerradas (CLRS, apéndice A, sección A.1)

$$\sum_{i=a}^b k = k(b - a + 1) \quad \sum_{i=1}^m i = \frac{m(m+1)}{2} \quad \sum_{i=0}^{n-1} i = \frac{(n-1)n}{2}$$

$$\sum_{i=0}^{n-1} i^2 = \frac{(n-1)n(2n-1)}{6}$$

$$\sum (x_i + y_i) = \sum x_i + \sum y_i \quad \sum k x_i = k \sum x_i$$

La rutina para cada ejercicio I

Cinco pasos, siempre los mismos

- 1 Entender: ¿qué entra, qué sale?
- 2 Trazar con un valor pequeño.
- 3 Tabular las vueltas de cada ciclo.
- 4 Reconocer el patrón y escribirlo como sumatoria.
- 5 Cerrar la suma y comprobar contra la traza.

Todos los ejercicios de hoy se resuelven con esa rutina. Cambia el código, no el camino.

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento
- 7 Todo junto
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Ejercicio 1 I

En parejas: escriba $T(n)$. Suponga $n \geq 1$.

```
int promedio(int datos[], int n) {  
    int suma = 0;  
    int i = 0;  
    while (i < n) {  
        suma = suma + datos[i];  
        i = i + 1;  
    }  
    return suma / n;  
}
```

Ejercicio 1: respuesta I

Línea	Veces
<code>int suma = 0;</code>	1
<code>int i = 0;</code>	1
<code>while (i < n)</code>	$n + 1$
<code>suma = suma + datos[i];</code>	n
<code>i = i + 1;</code>	n
<code>return suma / n;</code>	1

$$T(n) = 3n + 4.$$

El mismo esqueleto de `suma_arreglo`: un recorrido completo, costo lineal.

Ejercicio 2 I

Hay dos condicionales adentro del ciclo. En parejas: escriba $T(n)$.
¿Tiene mejor y peor caso?

```
int diferencia(int datos[], int n) {  
    int pares = 0;  
    int impares = 0;  
    int i = 0;  
    while (i < n) {  
        if (datos[i] % 2 == 0) {  
            pares = pares + 1;  
        }  
        if (datos[i] % 2 != 0) {  
            impares = impares + 1;  
        }  
        i = i + 1;  
    }  
    return pares - impares;  
}
```

Ejercicio 2: respuesta I

Con k valores pares en el arreglo:

Línea	Veces
Inicializaciones	3
while (i < n)	$n + 1$
if (datos[i] % 2 == 0)	n
pares = pares + 1;	k
if (datos[i] % 2 != 0)	n
impares = impares + 1;	$n - k$
i = i + 1;	n
return pares - impares;	1

$$T(n) = 3 + (n + 1) + n + k + n + (n - k) + n + 1 = 5n + 5.$$

Ejercicio 2: respuesta II

La k desaparece

Cada valor es par o impar: las dos asignaciones se reparten las n vueltas (k y $n - k$ suman n). Hay condicionales, pero no hay mejor ni peor caso.

Ejercicio 3 I

La condición es compuesta. En parejas: trace con $\{3, 4, 5\}$ y con $\{-1, 4, 5\}$, y encuentre el mejor y el peor caso.

```
int todos_positivos(int datos[], int n) {  
    int todos = 1;  
    int i = 0;  
    while (i < n && todos == 1) {  
        if (datos[i] <= 0) {  
            todos = 0;  
        }  
        i = i + 1;  
    }  
    return todos;  
}
```

Ejercicio 3: respuesta I

Con $\{-1, 4, 5\}$ el ciclo muere en la primera vuelta:

Evaluación	i	$i < n?$	$i \text{ todos} = 1?$	Acción
1	0	cierto	cierto	$\text{todos} = 0, i = 1$
2	1	cierto	falso	sale del ciclo

- Mejor caso: el primer valor no es positivo.

$$T = 2 + 2 + 1 + 1 + 1 + 1 = 8, \text{ constante.}$$

- Peor caso: todos son positivos, el ciclo recorre completo y la asignación nunca corre. $T(n) = 3n + 4$, lineal.

Se reporta el peor caso: es la garantía con cualquier entrada.

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia**
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento
- 7 Todo junto
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Ejercicio 4 I

El índice avanza de dos en dos. En parejas: ¿cuántas vueltas da el ciclo con $n = 6$ y con $n = 7$? Escriba $T(n)$.

```
int de_dos_en_dos(int datos[], int n) {  
    int suma = 0;  
    int i = 0;  
    while (i < n) {  
        suma = suma + datos[i];  
        i = i + 2;  
    }  
    return suma;  
}
```

Ejercicio 4: respuesta I

n	Valores de i	Vueltas
6	0, 2, 4	3
7	0, 2, 4, 6	4

Las vueltas son $\lceil n/2 \rceil$: la mitad, redondeada hacia arriba.

$$T(n) = 3 \left\lceil \frac{n}{2} \right\rceil + 4.$$

Sigue siendo lineal: la mitad de la pendiente, el mismo ritmo.

Ejercicio 5 I

Ahora el índice baja. En parejas: trace con $n = 5$ y escriba $T(n)$.

```
int descendente(int n) {  
    int suma = 0;  
    int i = n;  
    while (i > 0) {  
        suma = suma + i;  
        i = i - 1;  
    }  
    return suma;  
}
```

Ejercicio 5: respuesta I

Con $n = 5$: i toma 5, 4, 3, 2, 1 y la condición falla con $i = 0$. Cinco vueltas, seis evaluaciones.

$$T(n) = 3n + 4.$$

Bajar de n a 1 cuesta lo mismo que subir de 1 a n : lo que importa es cuántos valores recorre el índice, no hacia dónde va. De paso: descendente(5) devuelve 15, la suma de Gauss $\frac{5 \cdot 6}{2}$ del miércoles.

Ejercicio 6 I

El paso depende de n . Suponga $n \geq 10$. En parejas: trace con $n = 20$, $n = 65$ y $n = 1000$. ¿Cuántas vueltas da el ciclo?

```
int paso_grande(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i <= n) {  
        cuenta = cuenta + 1;  
        i = i + n / 5;  
    }  
    return cuenta;  
}
```

Ejercicio 6: las trazas I

n	Paso $n/5$	Valores de i	Vueltas
20	4	0, 4, 8, 12, 16, 20	6
65	13	0, 13, 26, 39, 52, 65	6
1000	200	0, 200, 400, 600, 800, 1000	6

Seis vueltas con $n = 20$ y seis con $n = 1000$. ¿Siempre seis?

Ejercicio 6: casi siempre I

Con $n = 14$ el paso es $14/5 = 2$ (división entera) y salen ocho vueltas: $i = 0, 2, 4, \dots, 14$.

El redondeo hacia abajo puede regalar una o dos vueltas más, pero con $n \geq 10$ nunca pasan de ocho: el paso mide cerca de la quinta parte de n , y en unas cinco o seis zancadas se cruza la meta.

$$T(n) \leq 3 \cdot 8 + 4 = 28 \quad \text{para todo } n \geq 10.$$

El costo no crece con n : el ciclo parece depender de n y es constante.

El caso límite

Con $n < 5$ la división entera da paso 0, i nunca avanza y el ciclo no termina. Un paso calculado siempre se revisa: ¿puede valer cero?

Ejercicio 7 I

El índice ahora se parte por la mitad. En parejas: trace con $n = 20$ y $n = 1000$. ¿Cuántas vueltas da el ciclo?

```
int mitades(int n) {  
    int i = n;  
    int cuenta = 0;  
    while (i > 0) {  
        cuenta = cuenta + 1;  
        i = i / 2;  
    }  
    return cuenta;  
}
```

Ejercicio 7: respuesta I

n	Valores de i	Vueltas
20	20, 10, 5, 2, 1	5
1000	1000, 500, 250, 125, 62, 31, 15, 7, 3, 1	10

Las mismas vueltas de potencias del miércoles: partir por la mitad es duplicar recorrido al revés, y la cuenta la lleva el mismo logaritmo:

$$\text{vueltas} = \lfloor \log_2 n \rfloor + 1 \quad T(n) = 3\lfloor \log_2 n \rfloor + 7.$$

Duplicar n añade una sola vuelta. El patrón reaparecerá cada vez que un algoritmo descarte la mitad de lo que le queda.

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes**
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento
- 7 Todo junto
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Ejercicio 8 I

En parejas, con la rutina completa: patrón, sumatoria y $T(n)$.

```
int prefijos(int datos[], int n) {  
    int total = 0;  
    int i = 0;  
    while (i < n) {  
        int j = 0;  
        while (j <= i) {  
            total = total + datos[j];  
            j = j + 1;  
        }  
        i = i + 1;  
    }  
    return total;  
}
```

Ejercicio 8: respuesta I

Con i congelado, j recorre $0, \dots, i$: el patrón es $1, 2, 3, \dots, n$ y el cuerpo corre $\sum_{i=0}^{n-1} (i+1) = \frac{n(n+1)}{2}$ veces.

Línea	Veces
<code>int total = 0;</code>	1
<code>int i = 0;</code>	1
<code>while (i < n)</code>	$n + 1$
<code>int j = 0;</code>	n
<code>while (j <= i)</code>	$\sum_{i=0}^{n-1} (i+2) = \frac{n(n+3)}{2}$
<code>total = total + datos[j];</code>	$\frac{n(n+1)}{2}$
<code>j = j + 1;</code>	$\frac{n(n+1)}{2}$
<code>i = i + 1;</code>	n
<code>return total;</code>	1

$$T(n) = \frac{3n^2 + 11n}{2} + 4 \quad T(1) = \frac{14}{2} + 4 = 11 \checkmark$$

Ejercicio 9 I

Las parejas estrictas: j arranca en $i + 1$. En parejas: patrón, sumatoria y $T(n)$.

```
int parejas_estrictas(int datos[], int n) {
    int suma = 0;
    int i = 0;
    while (i < n) {
        int j = i + 1;
        while (j < n) {
            suma = suma + datos[i] * datos[j];
            j = j + 1;
        }
        i = i + 1;
    }
    return suma;
}
```

Ejercicio 9: respuesta I

Con i congelado, j recorre $i + 1, \dots, n - 1$: son $n - i - 1$ vueltas.
El patrón baja: $n - 1, n - 2, \dots, 1, 0$.

$$\sum_{i=0}^{n-1} (n - i - 1) = \frac{(n - 1) n}{2}$$

$$T(n) = \frac{3n^2 + 5n}{2} + 4.$$

El mismo T de triángulo del miércoles: es el mismo triángulo, desplazado una diagonal.

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados**
- 6 Ponerle nombre al crecimiento
- 7 Todo junto
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

El reto I

En equipos: ¿cuántas veces corre `cuenta = cuenta + 1`?
Empiece trazando con $n = 4$ y $n = 5$.

```
int tercetos(int n) {  
    int cuenta = 0;  
    int i = 0;  
    while (i < n) {  
        int j = 0;  
        while (j < i) {  
            int k = 0;  
            while (k < j) {  
                cuenta = cuenta + 1;  
                k = k + 1;  
            }  
            j = j + 1;  
        }  
        i = i + 1;  
    }  
    return cuenta;  
}
```

El reto II

```
}
```

El reto: las trazas I

$$\text{tercetos}(4) = 4 \quad \text{tercetos}(5) = 10.$$

Cada ejecución del cuerpo corresponde a un trío de índices $k < j < i$: con $n = 4$ hay 4 formas de escoger tres posiciones distintas entre $\{0, 1, 2, 3\}$; con $n = 5$ hay 10.
¿Cómo se cuenta en general, con la rutina?

El reto: congelar i !

Con i congelado, los dos ciclos de adentro son exactamente `triangulo(i)`: su cuerpo corre

$$\frac{i(i-1)}{2} \text{ veces.}$$

El total es la suma de ese aporte sobre todas las vueltas del externo:

$$\text{cuenta} = \sum_{i=0}^{n-1} \frac{i(i-1)}{2}.$$

El trabajo del miércoles se volvió una pieza: lo que era un problema completo hoy es un término de la suma.

El reto: cerrar la suma I

Linealidad y formulario:

$$\begin{aligned}\sum_{i=0}^{n-1} \frac{i(i-1)}{2} &= \frac{1}{2} \left(\sum_{i=0}^{n-1} i^2 - \sum_{i=0}^{n-1} i \right) \\ &= \frac{1}{2} \left(\frac{(n-1)n(2n-1)}{6} - \frac{(n-1)n}{2} \right) \\ &= \frac{(n-1)n}{12} ((2n-1) - 3) = \frac{n(n-1)(n-2)}{6}.\end{aligned}$$

La prueba de siempre

$$n = 4: \frac{4 \cdot 3 \cdot 2}{6} = 4 \quad \checkmark \qquad n = 5: \frac{5 \cdot 4 \cdot 3}{6} = 10 \quad \checkmark$$

El cuerpo crece como $\frac{n^3}{6}$: tres niveles de dependencia producen un costo cúbico.

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento**
- 7 Todo junto
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

La cosecha de la semana I

Función	T	Término que manda
paso_grande	a lo sumo 28	constante
mitades	$3\lfloor \log_2 n \rfloor + 7$	$\log_2 n$
promedio	$3n + 4$	n
de_dos_en_dos	$3\lceil n/2 \rceil + 4$	n
prefijos	$\frac{3n^2+11n}{2} + 4$	n^2
tercetos	cerca de $\frac{n^3}{6}$	n^3

Cinco ritmos distintos. Para compararlos basta el término que manda.

El término que manda

En triángulo, $T(n) = \frac{3n^2+5n}{2} + 4$. Con $n = 1000$:

$$T(1000) = 1\,502\,504 \quad \frac{3}{2}n^2 = 1\,500\,000.$$

El término cuadrático pone el 99.8 % del costo: para n grande, el resto de la fórmula es equipaje.

La notación O

Lectura informal (CLRS, sección 3.1)

$T(n) \in O(n^2)$ se lee “ T crece a lo sumo como n^2 ”: quitando constantes y términos menores, n^2 es el techo del ritmo de crecimiento. La definición precisa llega con las demostraciones formales del curso.

Nombre	Se escribe	Ejemplo de hoy
Constante	$O(1)$	paso_grande
Logarítmico	$O(\log n)$	mitades
Lineal	$O(n)$	promedio
Cuadrático	$O(n^2)$	prefijos
Cúbico	$O(n^3)$	tercetos

Ahora usted!

Clasifique cada costo con su O :

(a) $7n + 2$ (b) 5 (c) $\frac{n(n+1)}{2}$ (d) $2\lfloor \log_2 n \rfloor + 9$

Ahora usted: respuesta I

(a) $O(n)$ (b) $O(1)$ (c) $O(n^2)$ (d) $O(\log n)$

En (c), el producto $n(n + 1)$ esconde un n^2 : siempre se expande antes de clasificar.

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento
- 7 Todo junto**
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Todo junto I

Una sola función con todo lo de la semana adentro. Los dos primeros bloques:

```
int combinado(int datos[], int n) {  
    int mayor = datos[0];  
    int i = 1;  
    while (i < n) {  
        if (datos[i] > mayor) {  
            mayor = datos[i];  
        }  
        i = i + 1;  
    }  
    int parejas = 0;  
    i = 0;  
    while (i < n) {  
        int j = i + 1;  
        while (j < n) {  
            if (datos[i] + datos[j] > mayor) {  
                parejas = parejas + 1;  
            }  
            j = j + 1;  
        }  
    }  
}
```

Todo junto II

```
    }  
    i = i + 1;  
}
```

Todo junto (continuación) I

Y el tercero:

```
int pasos = 0;
int valor = 1;
while (valor <= n) {
    pasos = pasos + 1;
    valor = valor * 2;
}
return parejas + pasos;
}
```

En equipos: ¿por dónde empezar? Suponga $n \geq 1$.

Por bloques I

Los tres bloques están en secuencia: sus costos se suman. Y cada uno se cuenta con su propia herramienta:

- Bloque 1: recorrido lineal con un `if` adentro.
- Bloque 2: ciclo dependiente, sumatoria y Gauss.
- Bloque 3: índice que se duplica, logaritmo.

Bloque 1: el mayor I

Línea	Veces
<code>int mayor = datos[0];</code>	1
<code>int i = 1;</code>	1
<code>while (i < n)</code>	n
<code>if (datos[i] > mayor)</code>	$n - 1$
<code>mayor = datos[i];</code>	k
<code>i = i + 1;</code>	$n - 1$

El índice arranca en 1, así que la condición se evalúa n veces. La asignación corre k veces, con k entre 0 (el primer valor es el mayor) y $n - 1$ (arreglo ascendente). Subtotal:

$$T_1 = 3n + k \quad \text{peor caso: } T_1 = 4n - 1.$$

Bloque 2: las parejas I

El mismo triángulo de parejas_estrictas, ahora con un if adentro que corre p veces (las parejas que superan al mayor):

Línea	Veces
<code>int parejas = 0;</code>	1
<code>i = 0;</code>	1
<code>while (i < n)</code>	$n + 1$
<code>int j = i + 1;</code>	n
<code>while (j < n)</code>	$\sum_{i=0}^{n-1} (n - i) = \frac{n(n+1)}{2}$
<code>if (...)</code>	$\frac{(n-1)n}{2}$
<code>parejas = parejas + 1;</code>	p
<code>j = j + 1;</code>	$\frac{(n-1)n}{2}$
<code>i = i + 1;</code>	n

Con p entre 0 y $\frac{n(n-1)}{2}$. En el peor caso (todas las parejas superan al mayor):

Bloque 2: las parejas II

$$T_2 = 2n^2 + 2n + 3.$$

Bloque 3: las duplicaciones I

Es potencias del miércoles con otro nombre: valor se duplica hasta pasar de n , y con el return al final,

$$T_3 = 3\lfloor \log_2 n \rfloor + 7.$$

El total I

Los tres subtotales se suman. En el peor caso:

$$T(n) = (4n - 1) + (2n^2 + 2n + 3) + (3\lfloor \log_2 n \rfloor + 7)$$

$$T(n) = 2n^2 + 6n + 3\lfloor \log_2 n \rfloor + 9.$$

Y en el mejor caso ($k = 0$ y $p = 0$; por ejemplo, todos los valores negativos):

$$T(n) = \frac{3n^2 + 11n}{2} + 3\lfloor \log_2 n \rfloor + 10.$$

La prueba de siempre

Con $n = 1$ no hay parejas ni vueltas del primer ciclo: las dos fórmulas coinciden y dan 17. Contado a mano, bloque por bloque: $3 + 7 + 7 = 17$.

El veredicto I

- El mejor y el peor caso difieren en las constantes, pero los dos son cuadráticos: el `if` no cambia el ritmo.
- El término que manda, n^2 , lo pone el bloque 2. El recorrido lineal y el logaritmo quedan de equipaje.
- Se reporta el peor caso: $T(n) \in O(n^2)$.

La regla que resume la semana

Una función con bloques en secuencia se analiza bloque por bloque, cada uno con su herramienta, y el bloque más costoso decide el O de toda la función.

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento
- 7 Todo junto
- 8 Ejercicio de cierre**
- 9 Resumen y referencias
- 10 Para practicar en casa

Ejercicio de cierre I

La condición frena al índice antes de llegar a n . Trace con $n = 10$, $n = 20$ y $n = 50$, proponga el patrón de las vueltas y escriba un $T(n)$ aproximado.

```
int frenado(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i * i < 2 * n) {  
        cuenta = cuenta + 1;  
        i = i + 1;  
    }  
    return cuenta;  
}
```


Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento
- 7 Todo junto
- 8 Ejercicio de cierre
- 9 Resumen y referencias**
- 10 Para practicar en casa

Resumen I

Lo que queda de la sesión

- La rutina no cambia: entender, trazar pequeño, tabular, reconocer el patrón, sumar y comprobar.
- El paso de k en k divide las vueltas; bajar cuesta lo mismo que subir; un paso proporcional a n deja las vueltas constantes.
- Duplicar el índice o partirlo por la mitad da $\lfloor \log_2 n \rfloor + 1$ vueltas: duplicar n añade una sola.
- Un nivel de dependencia da un triángulo ($\approx n^2/2$); dos niveles dan $\approx n^3/6$.
- El término que más crece decide el ritmo; la notación O le pone nombre: $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$, $O(n^3)$.
- Bloques en secuencia: se cuenta cada uno con su herramienta y el más costoso decide el O .

Referencias I



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein.
Introduction to Algorithms. 4.^a ed., MIT Press, 2022.
Secciones 2.2 y 3.1; apéndice A, sección A.1.



R. Thareja. *Data Structures Using C*. Oxford University Press,
2018. Capítulo 2.

Contenido I

- 1 La rutina de trabajo
- 2 Ciclos simples
- 3 El paso cambia
- 4 Ciclos dependientes
- 5 El reto: tres ciclos anidados
- 6 Ponerle nombre al crecimiento
- 7 Todo junto
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Propuesto 1 I

¿Cuántas vueltas da el ciclo? Escriba $T(n)$.

```
int descendente_tres(int n) {  
    int cuenta = 0;  
    int i = n;  
    while (i > 0) {  
        cuenta = cuenta + 1;  
        i = i - 3;  
    }  
    return cuenta;  
}
```

Propuesto 2 I

El externo salta de dos en dos y el interno es completo. Suponga n par. ¿Cuántas veces corre el cuerpo interno? Escriba $T(n)$.

```
int doble_salto(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i < n) {  
        int j = 0;  
        while (j < n) {  
            cuenta = cuenta + 1;  
            j = j + 1;  
        }  
        i = i + 2;  
    }  
    return cuenta;  
}
```

Propuesto 3 I

El interno llega hasta $2i$, incluido. ¿Cuántas veces corre el cuerpo interno? Escriba $T(n)$.

```
int doble_limite(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i < n) {  
        int j = 0;  
        while (j <= 2 * i) {  
            cuenta = cuenta + 1;  
            j = j + 1;  
        }  
        i = i + 1;  
    }  
    return cuenta;  
}
```

Pistas de los propuestos I

- **Propuesto 1:** bajar de tres en tres cuesta lo mismo que subir de tres en tres: $\lceil n/3 \rceil$ vueltas y $T(n) = 3\lceil n/3 \rceil + 4$. Sigue en $O(n)$.
- **Propuesto 2:** el interno es independiente, así que multiplica; solo cambian las vueltas externas. El cuerpo corre $\frac{n}{2} \cdot n = \frac{n^2}{2}$ veces y $T(n) = \frac{3n^2+4n}{2} + 4$: mitad de constante, mismo $O(n^2)$.
- **Propuesto 3:** el cuerpo corre $2i + 1$ veces por vuelta; separe con linealidad: $\sum(2i + 1) = 2\sum i + \sum 1 = n(n-1) + n = n^2$. La suma de los primeros n impares es un cuadrado perfecto, y $T(n) = 3n^2 + 4n + 4$, el mismo del anidado completo.

El ejercicio de cierre queda para resolver con la rutina de la sesión.